

Backend Roadmap

A structured path from programming fundamentals to advanced backend engineering

6 STAGES · FUNDAMENTALS → ADVANCED · SELF-PACED



Roadmap Overview

PROGRAMMING FUNDAMENTALS → DATABASES → NETWORKING → GIT & GITHUB → BACKEND FUNDAMENTALS → ADVANCED

01

Programming Fundamentals

02

Databases

03

Networking Basics

04

Git & GitHub

05

Backend Fundamentals

06

Advanced Topics

Programming Fundamentals

This stage builds the programming foundation every backend developer needs: thinking in code, breaking problems into logical steps, and writing structured programs before touching any framework.

WHAT YOU'LL LEARN

Programming Basics & Problem Solving

Core building blocks, and how to reason through problems step by step.

Variables & Data Types · Conditions · Loops · Functions · Basic Problem Solving

Core Concepts & C++

Model real-world entities, organize data efficiently, and practice in a strongly-typed language.

OOP · Data Structures · C++

LEARNING FLOW

Syntax → Logic → Problem Solving → OOP → Data Structures → C++

CHECKPOINT Can solve basic problems without relying on tutorials.

ESTIMATED TIME

6–7 Weeks

PROJECT

Console App

PRACTICE

- Solve progressively harder problems
- Implement basic data structures manually
- Build a small C++ console application

[RESOURCE / URL PLACEHOLDER]

Databases

Backend applications constantly store, retrieve, update, and relate data. This stage covers how to design a database and interact with it confidently using SQL.

WHAT YOU'LL LEARN

Database Types & SQL Basics

Compare structured and unstructured storage, then query relational data using CRUD, filtering, joins, and aggregation.

Relational Databases · NoSQL Databases · CRUD · SELECT / WHERE · JOIN · GROUP BY

Structure & Integrity

Model relationships and enforce data consistency.

Tables & Relationships · Primary / Foreign Keys · Constraints

LEARNING FLOW

Data → Tables → Relationships → SQL → Queries → App Database

CHECKPOINT Can design a schema and write multi-table SQL queries.

ESTIMATED TIME

2–3 Weeks

PROJECT

Schema + SQL

PRACTICE

- Design a schema for a real app — blog, bookstore, task manager
- Create tables and constraints
- Write queries involving JOIN and GROUP BY
- Connect an application to the database

[RESOURCE / URL PLACEHOLDER]

SELECT +
FROM users
WHERE id = 1;

Networking Basics

Backend development depends on understanding how clients and servers communicate. This stage covers the request/response cycle behind every API call.

WHAT YOU'LL LEARN

Foundations

Understand how machines find and address each other on a network.

Client / Server · IP & Ports · DNS · HTTP / HTTPS

Requests & Responses

Learn how HTTP messages carry data, intent, and status between client and server.

Request / Response · HTTP Methods · Status Codes · Headers

LEARNING FLOW

Client → DNS → IP / Port → HTTP → Request → Response

CHECKPOINT Can explain an HTTP request from client to server.

ESTIMATED TIME

2–3 Weeks

PROJECT

HTTP / API Inspection

PRACTICE

- Inspect real requests with browser DevTools
- Use curl or Postman
- Send GET / POST / PUT / DELETE requests
- Trace a browser-to-API request

DNS

TCP/IP

http://

[RESOURCE / URL PLACEHOLDER]

Git & GitHub

Version control keeps backend projects safe and makes collaboration possible. This stage covers the everyday Git workflow used on real teams.

WHAT YOU'LL LEARN

Git Basics

Track changes to code with commits and history.

Repository · Commit · Branches · Merge

Collaboration

Work with others without overwriting each other's changes.

Remote Repositories · Pull Requests · Conflict Resolution

LEARNING FLOW

Local Repo → Commit → Branch → Merge → Remote → Pull Request

CHECKPOINT Can work with branches, PRs, and resolve conflicts.

ESTIMATED TIME

1 Week

PROJECT

Collaborative Repo

PRACTICE

- Build a small project using Git from day one
- Practice feature branches and meaningful commits
- Open pull requests
- Resolve a merge conflict

[RESOURCE / URL PLACEHOLDER]

Backend Fundamentals — Frameworks & Architecture

Frameworks provide structure and reusable tools for building backend services. Architecture is how a request actually moves through that structure, from arrival to response.

WHAT YOU'LL LEARN

Backend Frameworks

Pick one stack and build with it, rather than learning every framework at once.

Python · Django · FastAPI · Other frameworks/ languages

Backend Architecture

Understand how a request is routed, processed, and returned.

Routing · Request Lifecycle · Middleware · Validation · Error Handling · Database Integration

LEARNING FLOW

Request → Router → Middleware → Logic → Validation → Response

CHECKPOINT Can explain how a request moves from route to response.

ESTIMATED TIME

2–4 Weeks

PROJECT

First Backend Service

PRACTICE

- Choose one backend stack and build with it
- Build an API and trace a request end to end
- Add basic validation and error handling

[RESOURCE / URL PLACEHOLDER]

Backend Fundamentals — API Development

APIs are the main interface through which frontend applications and other services communicate with the backend — this stage turns architecture knowledge into a real, usable API.

WHAT YOU'LL LEARN

REST APIs

Structure endpoints around resources and standard HTTP conventions.

REST APIs · CRUD Operations · API Structure

Query Features & Documentation

Let clients navigate and shape the data they receive, then document it for others.

Pagination · Filtering · Sorting · API Documentation

LEARNING FLOW

Checkpoint → CRUD → Filter / Sort → Paginate → Document

CHECKPOINT Can build and document a working REST API.

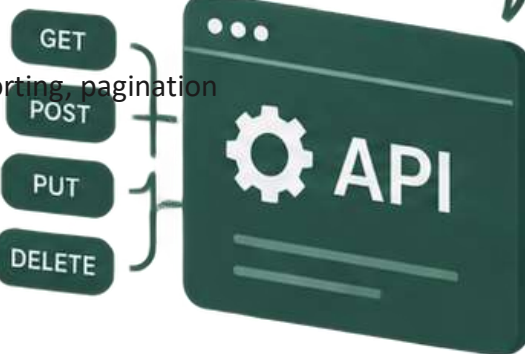
PRACTICE

- Build a Task Management API
- Add CRUD endpoints, filtering, sorting, pagination
- Document the finished API

ESTIMATED TIME
2–3 Weeks

PROJECT
REST API

HTTP



[RESOURCE / URL PLACEHOLDER]

Authentication & Authorization

Authentication proves who a user is; authorization decides what that user is allowed to do. Almost every real backend needs both.

WHAT YOU'LL LEARN

Core Concepts

Separate identity verification from access control.

Authentication vs Authorization · Password Hashing

Implementation

Issue and check credentials on every protected request.

Sessions / Tokens · JWT Basics · Roles & Permissions

LEARNING FLOW

Credentials → Hashing → Authentication → Token / Session → Authorization

CHECKPOINT Can implement protected endpoints and permissions.

ESTIMATED TIME
1–2 Weeks

PROJECT
Authenticated API

PRACTICE

- Implement registration and login
- Hash passwords before storing them
- Add token or session authentication
- Protect endpoints with roles and permissions

[RESOURCE / URL PLACEHOLDER]



Advanced Topics

These are deeper disciplines to study once the core backend foundation is strong. Each is worth exploring on its own — this stage is advanced and ongoing, not a fixed checklist.

ESTIMATED TIME
6–10+ Wks

STATUS
Ongoing

Security

Protect APIs, credentials, data, and application boundaries.

Testing

Verify backend behavior stays correct as the system evolves.

Deployment

Move the application from local development to a real environment.

Cloud

Understand the infrastructure and managed services that run it.

DevOps

Automate builds, testing, and delivery with CI/CD and containers.

System Design

Design backend systems that scale and stay maintainable.

LEARNING FLOW

Build → Test → Secure → Deploy → Operate → Scale

CHECKPOINT Can deploy, test, secure, and reason about a backend system.

PROJECT Deployed Production-like Backend

Backend Roadmap Summary

THE FULL LEARNING PROGRESSION

- 01 **Programming Fundamentals**
- ↓
- 02 **Databases**
- ↓
- 03 **Networking Basics**
- ↓
- 04 **Git & GitHub**
- ↓
- 05 **Backend Fundamentals**
- ↓
- 06 **Authentication & Authorization**
- ↓
- 07 **Advanced Backend Engineering**

